МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ

ҮЙЛДЛИЙН СИСТЕМИЙН ОНОЛ

Лабораторийн ажил 1

С хэл дээрх энгийн командын мөрийн интерфейс

Гүйцэтгэсэн оюутан _____

Оюутны код _____

Шалгасан багш _____

Улаанбаатар хот
2026 он

1 Ажлын зорилго

Энэ лабораторийн ажлын зорилго нь командын мөрийн интерфейс хэрхэн ажилладгийг ойлгож, түгээмэл хэрэглэгддэг файл болон директорын командуудыг C хэлээр өөрсдөө хэрэгжүүлэх явдал юм. Программ нь үйлдлийн системийн бэлэн shell командыг system() эсвэл popen()-оор дуудахгүй. Харин C болон POSIX сангийн функцүүдийг шууд ашиглана.

2 Даалгаврын шаардлага

- dir - одоогийн директорын файл, дэд директоруудыг харуулах
- create - шинэ хоосон файл үүсгэх
- input - файлд текст өгөгдөл нэмэх
- rename - файлын нэр өөрчлөх
- del - файл устгах
- cd - одоогийн ажлын директорыг өөрчлөх

Хэрэглэхэд ойлгомжтой болгохын тулд pwd, help, exit гэсэн нэмэлт командуудыг оруулсан.

3 Ашигласан орчин

Үзүүлэлт	Утга
Үйлдлийн систем	Linux
Програмчлалын хэл	C
Хөрвүүлэгч	GCC
Стандарт	C11
Гол сангууд	stdio.h, dirent.h, unistd.h, sys/stat.h

4 Программын бүтэц ба ажиллах зарчим

Программ эхлэхэд давталт ажиллаж, хэрэглэгчээс нэг мөр команд авна. Оруулсан мөрийг strtok() функцээр команд болон аргумент болгон салгана. Командын нэрийг strcmp()-ээр шалгаж, тохирох функцийг дуудна. exit команд орж ирэхэд давталт дуусна.

Үндсэн алгоритм

1. Программыг эхлүүлж тусламжийн богино мэдээлэл харуулна.
2. os> prompt хэвлээд хэрэглэгчийн мөрийг fgets() ашиглан уншина.
3. Командын нэр болон аргументуудыг салгана.
4. Тохирох функцийг ажиллуулж, үр дүн эсвэл алдааны мэдээллийг хэвлэнэ.
5. exit команд хүртэл 2-4 дүгээр алхмыг давтана.

5 Командуудын хэрэгжүүлэлт

Команд	Ашигласан функц	Тайлбар
dir	opendir, readdir, stat	Одоогийн директорын бүх элементийг уншиж, директорыг [DIR] гэж тэмдэглэнэ.
create	fopen (wx)	Өмнө нь байгаагүй шинэ хоосон файл үүсгэнэ. Давхар файлыг дарж бичихгүй.
input	fopen (a), fprintf	Файлыг append горимоор нээж, оруулсан текстийг шинэ мөр болгон нэмнэ.
rename	rename	Хуучин нэрийг шинэ нэрээр солино.
del	unlink	Зөвхөн файлыг устгана.
cd	chdir	Процессын одоогийн ажлын директорыг солино.
pwd	getcwd	Одоогийн ажлын директорын замыг харуулна.

6 Хөрвүүлэх ба ажиллуулах

Лабораторийн хавтсанд дараах командыг ажиллуулна.

```
make
./os_shell
```

Программыг дуусгахдаа exit команд өгнө. Хөрвүүлсэн файлыг цэвэрлэхдээ make clean ашиглана.

7 Туршилт ба үр дүн

Программыг GCC-ийн -Wall -Wextra анхааруулгатайгаар хөрвүүлсэн. Дараах туршилтуудаар бүх үндсэн командыг шалгав.

№	Оруулсан команд	Хүлээгдсэн үр дүн	Төлөв
1	create text.txt	Шинэ хоосон файл үүснэ	Амжилттай
2	input text.txt sample data	Файлд sample data мөр нэмэгдэнэ	Амжилттай
3	dir	text.txt жагсаалтад харагдана	Амжилттай
4	rename text.txt file.txt	Файлын нэр file.txt болно	Амжилттай
5	del file.txt	Файл устгагдана	Амжилттай
6	cd .. болон pwd	Ажлын директор нэг түвшин дээшилнэ	Амжилттай
7	unknown	Танигдаагүй командын мэдэгдэл гарна	Амжилттай

Туршилтын жишээ

```
os> create text.txt
Created: text.txt
os> input text.txt sample data
Data written to: text.txt
os> rename text.txt file.txt
Renamed: text.txt -> file.txt
os> del file.txt
Deleted: file.txt
```

8 Дүгнэлт

Энэ ажлаар shell программ хэрэглэгчийн командыг уншиж, задлаад тохирох системийн функцтэй холбодгийг ойлголоо. Файл болон директортой ажиллах үйлдлүүдийг бэлэн команд дуудахгүйгээр C хэлний функцүүдээр хэрэгжүүлсэн. Мөн буруу аргумент, байхгүй файл, давхар файл зэрэг энгийн алдааг шалгаж мэдээлэл хэвлэдэг болгосон.

9 Хязгаарлалт ба сайжруулах боломж

Энэ хувилбар нь лабораторийн үндсэн шаардлагад зориулсан тул нэг мөрөөр нэг команд ажиллуулна. Файлын нэр болон директорын замд хоосон зай байвал тусгай хашилт боловсруулахгүй. Цаашид давхар хашилттай аргумент, командын түүх, автоматаар гүйцээх болон файл хуулж зөөх команд нэмж өргөтгөж болно.

Алдаа боловсруулахдаа perror() ашигласан учраас байхгүй файл, буруу зам, эрхийн алдааны шалтгааныг үйлдлийн системээс ирсэн мэдээллээр харуулна. create команд өмнөх файлыг санамсаргүй дарахгүй, del команд директорыг устгахгүй байдлаар хийгдсэн.

Хавсралт А Эх код

os_shell.c файлын бүтэн эх кодыг мөрийн дугаартайгаар хавсаргав.

```
  1  #include <dirent.h>
  2  #include <errno.h>
  3  #include <stdio.h>
  4  #include <stdlib.h>
  5  #include <string.h>
  6  #include <sys/stat.h>
  7  #include <unistd.h>
  8
  9  #define INPUT_SIZE 1024
 10  #define PATH_SIZE 4096
 11
 12  static void remove_newline(char *text)
 13  {
 14      size_t length = strlen(text);
 15      if (length > 0 && text[length - 1] == '\n') {
 16          text[length - 1] = '\0';
 17      }
 18  }
 19
 20  static char *skip_spaces(char *text)
 21  {
 22      while (*text == ' ' || *text == '\t') {
 23          text++;
 24      }
 25      return text;
 26  }
 27
 28  static void command_dir(void)
 29  {
 30      DIR *directory = opendir(".");
 31      struct dirent *entry;
 32
 33      if (directory == NULL) {
 34          perror("dir");
 35          return;
 36      }
 37
 38      while ((entry = readdir(directory)) != NULL) {
 39          struct stat file_info;
 40
 41          if (strcmp(entry->d_name, ".") == 0 ||
 42              strcmp(entry->d_name, "..") == 0) {
 43              continue;
 44          }
 45
 46          if (stat(entry->d_name, &file_info) == 0 && S_ISDIR(file_info.st_mode)) {
 47              printf("[DIR]  %s\n", entry->d_name);
 48          } else {
 49              printf("       %s\n", entry->d_name);
 50          }
 51      }
 52
 53      closedir(directory);
 54  }
 55
 56  static void command_create(const char *filename)
 57  {
 58      FILE *file;
 59
 60      if (filename == NULL) {
 61          printf("Usage: create <filename>\n");
 62          return;
 63      }
 64
 65      file = fopen(filename, "wx");
 66      if (file == NULL) {
 67          if (errno == EEXIST) {
 68              printf("File already exists: %s\n", filename);
 69          } else {
 70              perror("create");
 71          }
 72          return;
 73      }
 74
 75      fclose(file);
 76      printf("Created: %s\n", filename);
 77  }
 78
 79  static void command_input(const char *filename, const char *data)
 80  {
 81      FILE *file;
 82
 83      if (filename == NULL || data == NULL || data[0] == '\0') {
 84          printf("Usage: input <filename> <data>\n");
 85          return;
```

```
    }

    file = fopen(filename, "a");
    if (file == NULL) {
        perror("input");
        return;
    }

    fprintf(file, "%s\n", data);
    fclose(file);
    printf("Data written to: %s\n", filename);
}

static void command_rename(const char *old_name, const char *new_name)
{
    if (old_name == NULL || new_name == NULL) {
        printf("Usage: rename <old_name> <new_name>\n");
        return;
    }

    if (rename(old_name, new_name) != 0) {
        perror("rename");
        return;
    }

    printf("Renamed: %s -> %s\n", old_name, new_name);
}

static void command_delete(const char *filename)
{
    if (filename == NULL) {
        printf("Usage: del <filename>\n");
        return;
    }

    if (unlink(filename) != 0) {
        perror("del");
        return;
    }

    printf("Deleted: %s\n", filename);
}

static void command_cd(const char *path)
{
    if (path == NULL) {
        printf("Usage: cd <directory>\n");
        return;
    }

    if (chdir(path) != 0) {
        perror("cd");
    }
}

static void command_pwd(void)
{
    char path[PATH_SIZE];

    if (getcwd(path, sizeof(path)) == NULL) {
        perror("pwd");
        return;
    }

    printf("%s\n", path);
}

static void print_help(void)
{
    printf("Available commands:\n");
    printf("  dir                         List current directory\n");
    printf("  create <filename>           Create a new empty file\n");
    printf("  input <filename> <data>     Append data to a file\n");
    printf("  rename <old> <new>          Rename a file\n");
    printf("  del <filename>              Delete a file\n");
    printf("  cd <directory>              Change current directory\n");
    printf("  pwd                         Show current directory\n");
    printf("  help                        Show this help\n");
    printf("  exit                        Exit the program\n");
}

int main(void)
{
    char line[INPUT_SIZE];

    printf("Simple OS Shell. Type 'help' to see commands.\n");

    while (1) {
        char *command;
        char *first_argument;
        char *second_argument;
```

```
177
178         printf("os> ");
179         fflush(stdout);
180
181         if (fgets(line, sizeof(line), stdin) == NULL) {
182             printf("\n");
183             break;
184         }
185
186         remove_newline(line);
187         command = strtok(line, " \t");
188         if (command == NULL) {
189             continue;
190         }
191
192         if (strcmp(command, "dir") == 0) {
193             command_dir();
194         } else if (strcmp(command, "create") == 0) {
195             command_create(strtok(NULL, " \t"));
196         } else if (strcmp(command, "input") == 0) {
197             first_argument = strtok(NULL, " \t");
198             second_argument = strtok(NULL, "");
199             if (second_argument != NULL) {
200                 second_argument = skip_spaces(second_argument);
201             }
202             command_input(first_argument, second_argument);
203         } else if (strcmp(command, "rename") == 0) {
204             first_argument = strtok(NULL, " \t");
205             second_argument = strtok(NULL, " \t");
206             command_rename(first_argument, second_argument);
207         } else if (strcmp(command, "del") == 0) {
208             command_delete(strtok(NULL, " \t"));
209         } else if (strcmp(command, "cd") == 0) {
210             command_cd(strtok(NULL, " \t"));
211         } else if (strcmp(command, "pwd") == 0) {
212             command_pwd();
213         } else if (strcmp(command, "help") == 0) {
214             print_help();
215         } else if (strcmp(command, "exit") == 0) {
216             break;
217         } else {
218             printf("Unknown command: %s\n", command);
219         }
220     }
221
222     return 0;
223 }
```

Хавсралт Б Makefile

```
1  CC = gcc
2  CFLAGS = -Wall -Wextra -std=c11
3  TARGET = os_shell
4
5  all: $(TARGET)
6
7  $(TARGET): os_shell.c
8  	$(CC) $(CFLAGS) os_shell.c -o $(TARGET)
9
10 clean:
11 	rm -f $(TARGET)
12
13 .PHONY: all clean
```